MR EMILE HARMEL

Chief Creative Technologist | Systems Architect | Founder
Graz, Austria | +43 680 154 2697 |
hello@hobolabs.digital
https://www.linkedin.com/in/emile-harmel-5bb238b4/

PROFESSIONAL SUMMARY

"I build worlds that make logic feel human — and humanity feel designed."

I come from a deep background in traditional **Frontend Development** and **UI/UX Design**, where pixel-perfect precision was the gold standard. Today, I treat **AI-Native Development** and **"Agentic Coding"** as the natural evolution of that craft.

Transitioning to an "Architect of Systems" doesn't mean losing quality; it means scaling it. I now conduct an orchestra of AI agents and automated workflows, ensuring that the soul of the design is never lost in the machinery. I bridge the gap between abstract vision and tangible reality, weaving design, code, psychology, and orchestration into cohesive digital ecosystems.

Highly experienced professional with over 20 years bridging complex backend architectures and intuitive, highly scalable user interfaces. Specializing in enterprise platform modernization, secure mobile ecosystems, and AI-driven automation workflows.

Core Expertise: Proven success in designing and implementing intuitive software solutions, leveraging Next.js, Python (FastAPI), system orchestration, and Atomic Design principles.

Industry Experience: Demonstrated track record in highly regulated environments (e.g., healthcare/ePA) and large-scale pan-European consumer platforms. Seeking to apply expertise to drive impactful technical innovation.

CORE COMPETENCIES

Strategic Leadership	**Creative Technology**	**Systems Engineering**
Product Founder Strategy	**Unreal Engine 5** (Blueprints/C++)	**Next.js 16 / React 19 / Vite**
Cross-Disciplinary Team Lead	**AI Agents & Orchestration** (Ollama, n8n, hermes)	**Python** (FastAPI, Pydantic)
Atomic Design Systems	**3D Worldbuilding & Texture Design**	**Microservices (Docker, Cloud Run)**
Business Logic & Negotiation	**Generative AI Workflows**	**Monorepo Architecture (Nx)**

PROFESSIONAL EXPERIENCE

Co-Founder & Chief Creative Technologist

MonstoryX | *2024 – Present*

The "Creative Spine and Technical Backbone" of an immersive digital storytelling platform.

- **Vision & Worldbuilding**: Defined the complete creative direction, from character lore and clay-style aesthetic to the "hero features" (Movie Maker, Monster Hub).
- **Technical Architecture**: Architected the entire **Unreal Engine 5** game system, including custom C++ plugins, mobile optimization for iOS/Android, and backend economy logic via Nakama.
- **Product Leadership**: Led the product roadmap from concept to deliver, balancing shareholder negotiations, grant funding, and cross-platform UX (Switch, Mobile, PC).
- **AI Innovation**: Designed multi-agent orchestration systems for dynamic storytelling and procedural content generation.

Lead Product Architect & Creative Technologist

Xitrust / Moxis Platform | *2024 – 2026*

Spearheaded the architectural reinvention of an enterprise platform, proving that complex data systems can feel fluid, intuitive, and human.

- **Design Engineering Strategy**: Built a comprehensive design system in **Figma**, leveraging a rigorous feedback loop: bouncing ideas from high-fidelity prototypes to Figma designs, and back to code-based prototypes to validate interaction models before final implementation.
- **Architecture**: Designed a scalable **monorepo architecture** unifying multiple apps with a shared **Atomic Design System**.
- **Full-Stack Modernization**: Built a next-gen frontend with **Next.js 16** and **React 19**, backed by a high-performance **Python FastAPI** service.
- **Vibe Coding**: Integrated **Local LLMs (Ollama)** to drive intelligent user workflows, transforming static forms into "AI-augmented" conversations.

iOS Developer

Rise World | *2021 – 2024*

- **Gematik e-Prescription App (ePA)**: Key developer for the official German e-prescription app, a high-stakes digital health solution used by millions.
- **Native iOS Development**: Specialized in **Swift** and Native iOS frameworks to build secure, compliant, and accessible mobile experiences within a strictly regulated healthcare environment.

Lead Creative Developer & Technical Director

Demodern – Creative Technologies | *2016 – 2021*

- **Nestlé Wagner (Spark AR)**: Developed viral Augmented Reality games and face filters for social media campaigns.
- **Mazda (Find My Mazda / My Mazda App)**: Lead Developer for Mazda's pan-European mobile ecosystem. Architected the "My Mazda" React Native app, handling critical service data and dealer integration across all EU markets.
- **Super RTL (Woozle Goozle)**: Created an innovative conversational interface for children using **Google Dialogflow**, blending NLP with character-driven UI.

TECHNICAL ARSENAL

- **Frontend Ecosystem**: React 19, Next.js 16 (App Router), Vite.js, Vue (Nuxt), TypeScript, MUI 7, Tailwind.
- **Backend & AI**: Python, FastAPI, Pydantic, Ollama (Local LLMs), n8n, OpenAI API, Google Dialogflow. Claud, Hermes

- **Mobile & Creative**: Swift (iOS Native), React Native, Unreal Engine 5, Spark AR, WebGL.
- **Cloud & DevOps**: Google Cloud Platform (Cloud Run), Docker, GitHub Actions/Pipelines, Nx Monorepo.
- **Design & Workflow**: Figma (Advanced Prototyping), IntelliJ IDEA, Storybook, Jira/Confluence. Claude Design

---

EDUCATION

City College Manchester

- **HND Multimedia Design**
- **ND Art Foundation with Mathematics**
- **City And Guild Project Management Certificate**

THE "VIBE"

- **Philosophy**: "Every system is a story — every story, a system in disguise."
- **Superpower**: **Systems Whisperer**. I turn tangled pipelines and abstract emotions into clean, living frameworks.
- **Tools**: Figma, Blender, Unreal Engine, Antigravity, Claude Code, VS Code (Cursor/Windsurf), Docker, Slack (Communication & Vibe Checks).

Case Study: How I Use AI Agents in My Daily Workflow

Building Moxis — An AI-Powered Document Intelligence Platform

Context

I run **Hobolabs Digital**, a solo consultancy where I design, build, and sell full-stack AI products. My current project is **Moxis**, a hybrid AI platform that automatically detects, classifies, and exports form fields from PDF contracts and legal documents. Think: given a 20-page lease agreement, where are the signature lines, date fields, checkboxes, and text inputs — and who is supposed to fill each one in?

The product spans four codebases in a monorepo:

- **Production app** (`apps/moxis-web`): Next.js 16 + MUI + Clerk auth + i18n
- **Technical demo** (`apps/demo-ui`): Full Inspector-Viewer-Sidebar UX with Training Console
- **AI backend** (`packages/ai-backend`): Python/FastAPI inference server orchestrating geometric CV + Vision Language Models
- **Training pipeline** (`packages/ai-trainer`): GPU worker for LoRA fine-tuning, model export, and deployment

It ships as a source-code license to enterprises (law firms, insurers) who run it entirely on their own infrastructure — no data leaves their cloud. The self-healing pipeline (Enterprise tier) lets the model adapt to each client's specific document templates over time.

How AI Agents Fit Into This

As a solo developer operating at this scope, AI agents aren't a toy — they're how I ship. Here's exactly how I use them.

---

1. Architecture Design Partner

When I started Moxis, I had a rough idea: geometric CV for fast cases, VLMs for hard ones, and an active learning loop to close the gap. But fleshing that out into a clean, modular architecture required pressure-testing.

I use Claude Code as a **design sparring partner**. I'll describe the constraints ("the geometric engine works at 72 DPI, but VLMs need 150 DPI images — how should coordinate scaling flow through the pipeline?") and the agent reads the actual codebase, proposes designs, and identifies edge cases I missed.

Concrete example: The **Visual Prompting v2** system. The idea was to annotate page images with bounding boxes and ID labels, send one image to the VLM, and have it return NDJSON for all fields in a single pass. The agent helped design the coordinate normalization scheme (0-1000 grid), proposed the anti-ghost rescue mechanism (fields the VLM silently skips get yielded as "unverified" rather than dropped), and flagged the truncation issue with Qwen models before it hit production. I didn't blindly accept the design — I challenged the streaming protocol choice (SSE vs WebSocket) — but having a capable thinking partner cuts architectural iteration from days to hours.

---

2. Full-File Refactoring Without the Drudgery

The codebase has undergone multiple major refactors as the architecture evolved. A recent one: extracting the training pipeline from the inference backend into its own `packages/ai-trainer` package with a separate Dockerfile and deploy script.

These aren't search-and-replace jobs — they involve cross-cutting changes across Python modules, TypeScript components, config files, and deployment scripts. I'll task the agent with the mechanical work:

> "Extract all training-related code from `packages/ai-backend/app/services/training_manager.py` and the scripts in `packages/ai-trainer/app/scripts/` into a clean standalone package. The backend should call it remotely via GCS + Cloud Run Jobs. Update both Dockerfiles and deploy scripts."

The agent reads the full context, proposes the file split, handles imports, and writes the boilerplate. I review the architecture decisions and focus on the parts that need human judgment: error handling strategy, retry semantics, when to block vs fire-and-forget.

---

3. Multi-File Feature Implementation

The **Enterprise Training Mode** feature required changes across:

- Backend: new `POST /training/start` endpoint, `TrainingManager` service, GCS upload
- Trainer: `train_remote.py` (download dataset, LoRA fine-tune, GGUF export)
- Demo UI: Training Console component, `QueueList`, API client wiring
- Config: new YAML sections, CORS origins, environment variables

As a solo dev, the bottleneck isn't typing — it's context switching between 6 files while holding the whole feature in your head. I use AI agents to handle the cross-cutting mechanical work while I focus on the decisions that matter: what's the state machine for training status? What happens when a training run crashes mid-export? How does the UI respond to SSE progress events?

The agent writes the boilerplate (endpoint scaffolding, GCS client, React state management). I write the tricky parts (streaming progress handler, LoRA merge logic) and review everything.

---

4. Deep-Dive Code Reviews

Before shipping the **StreamingVisionProvider v2**, I asked the agent to review it with a specific lens:

> "Review this provider for race conditions in the async streaming code. Specifically: are the semaphore-bound VLM calls safe under concurrent page processing? What happens if the model returns a truncated NDJSON line? Are the coordinate transform pipelines (72→150→1000→150→72) consistent in all error paths?"

This isn't about catching style issues — it's about having a second pair of eyes that can trace async control flow across 400 lines without losing context. The agent caught a case where a cancelled task wouldn't release the semaphore, and flagged that the hybrid tiling mode's coordinate offset logic needed a bounds check. These are exactly the bugs a human reviewer would find — the agent just reads the code far faster than one could.

---

5. Documentation & Technical Writing

The **Moxis Tech Pitch** (a 119-line sales document covering tiers, pricing, and architecture) started as bullet points. I described the product, the three tiers, and the technical highlights conversationally. The agent structured it into a polished document with clear sections, pricing tables, and deployment scenarios — written in the tone of a technical sales pitch aimed at enterprise buyers.

This pattern recurs constantly: I draft the substance, the agent handles the polish and structure. READMEs, API docs, deployment guides — the kind of writing that matters but doesn't justify hours of a solo dev's time.

---

6. Bash & Infrastructure as a Second Language

Cloud Run deployment configs, Dockerfile tuning, `deploy.sh` scripts — these are fiddly, error-prone, and slow to iterate on manually. Rather than looking up the `gcloud run deploy` flag for specifying NVIDIA L4 GPUs or the exact `pytorch/pytorch` Docker tag with CUDA 12.1, I describe what I need and the agent runs the commands, checks the output, and adjusts.

I still approve all cloud operations explicitly — the agent never pushes, deploys, or modifies infra without confirmation — but the feedback loop is dramatically faster than manual trial-and-error.

---

7. Active Learning Dataset Pipeline

The self-healing pipeline's dataset builder (`build_dataset.py`, `build_master_dataset.py`) converts user corrections from the Inspector UI into ShareGPT-format training data. Getting the schema right — mapping bounding box coordinates, field types, and page metadata into a format that LLaMA-Factory expects — required reading multiple format specifications and aligning them with our internal data model.

The agent handled the schema mapping research (reading LLaMA-Factory's expected format, llama.cpp's GGUF export conventions) and wrote the ingestion scripts. I focused on the data quality heuristics: when is a user correction trustworthy enough to include in the gold set? How do we handle conflicting corrections for the same field?

What I've Learned

After six months of integrating AI agents into a real production workflow, a few patterns are clear:

Agents amplify existing competence. The agent writes better code when I write better prompts. Vague instruction → vague output. Specific instruction with file paths, constraints, and edge cases → useful output. The agent doesn't replace engineering judgment; it removes the friction between having judgment and expressing it in code.

Delegation is a skill. Knowing what to delegate and what to own is the meta-skill. Boilerplate, cross-file wiring, format conversions, doc polish — delegate aggressively. Architecture decisions, error handling strategy, data quality heuristics, security boundaries — own them.

The multiplier is real. I ship at a pace that would normally require 2-3 engineers. Not because the agent does the thinking, but because it eliminates the mechanical drag that consumes most of a developer's time: looking up APIs, writing scaffolding, navigating between files, formatting documentation.

Review everything. The agent is wrong often enough to be dangerous. Every generated line gets reviewed — not skimmed, reviewed. The cost of review is dramatically lower than the cost of writing, so the net is still strongly positive.

This is the most productive I've been in 15 years of building software — not because the tools are magic, but because they let me spend a higher fraction of my time on the work that actually needs a human brain.

Case Study: Hermes — A Multi-Agent AI Operating System for Autonomous Content Production

Overview

Project: Hermes AI Infrastructure (MonstoryX) **Role:** AI Systems Architect & Lead Developer **Stack:** GCP (dual-VM serverless GPU), Python, ComfyUI, LTX-Video, Claude Code, Slack Block Kit

---

The Problem

MonstoryX is a generative AI-driven narrative platform for early childhood literacy. The content pipeline requires: researching trends, writing funding pitches, generating storyboard panels, producing animated shorts with consistent characters, and publishing to YouTube — all on a shoestring budget. Doing this manually would require a team of 5-6 people. The challenge was to build an autonomous system where AI agents handle the entire pipeline end-to-end, with humans only approving or rejecting work via Slack.

The Hermes Flow: A Hub-and-Spoke Agent Architecture

The core innovation is **Hermes** — a multi-agent orchestration system I architect on Google Cloud Platform. It follows a hub-and-spoke pattern:

The Orchestrator ("Herman") runs 24/7 on a low-cost VM. It hosts specialized AI profiles — Research, Creative, Marketing — each with its own tools, memory, and permissions. All human interaction happens through Slack: requests come in via department channels, agents process them autonomously, and output is routed back with interactive Approve/Reject/Re-run buttons.

The GPU Worker is a serverless ComfyUI instance with an NVIDIA L4 GPU. Here's the key design decision: it stays **powered off by default**. When the Creative agent needs to generate images or video, it programmatically boots the GPU VM via `gcloud compute`, does the work, and immediately shuts it down. This keeps GPU costs to roughly $0.50/hour of actual use rather than $300+/month of idle time.

The agents communicate over internal VPC networking (no public IPs on the GPU worker — defense in depth). A Python wrapper script (`manage_comfyui_vm.py`) and an SSH proxy (`comfy_wrapper.sh`) make the power cycling transparent to the agents.

What makes this an agent system rather than a script:

- Each profile has persistent memory (via Honcho, an agent memory layer with vector storage, observation derivation, and self-improving recall)
- Agents make autonomous decisions (which funding opportunity to pursue, which panel needs a re-run, when to escalate)
- Human-in-the-loop approval is async — the agent fires off an `ack()` to Slack in under 3 seconds, then dispatches work in a background thread
- Cron-scheduled jobs bypass approval entirely (`is_scheduled_job: True`), making daily research briefings fully autonomous

---

The Image-to-Video Pipeline: LTX-2.3 AV-LoRA

The current milestone (v1.3) pushes Hermes into video generation. The goal: from a single character reference image, generate a fully animated short with synchronized audio — all triggered by a Slack message.

Phase 1 — Auto-Captioning: Florence-3-Large vision model captions every training image with the trigger word `sks_Trio`. Critical constraint learned: all images must be multiples of 32 pixels, or LTX-Video produces ghosting artifacts from internal padding.

Phase 2 — Dual-LoRA Training inside ComfyUI: Two LoRAs are trained on the same dataset. A Flux.1 [dev] LoRA (Rank 16) learns to generate perfect start/end frames. An LTX-Video LoRA (Rank 64, 17 frames at 24fps) learns character motion consistency — ensuring the character's markings don't "slide" during animation. Training happens entirely inside ComfyUI via Ostris custom nodes, not standalone scripts, after we learned that standalone trainers break the node graph integration.

Phase 3 — Audio Fusion: Fish Speech S2 clones the character's voice, and an `LTXAudioSampler` node embeds both voice and foley (clay-scuffing sounds) directly into the model during training. The output is a single latent that separates into synchronized video + audio.

Phase 4 — Continuous Validation: Every 200 training steps, a `Validation_Sampler` generates a 2-second preview clip with the prompt *"sks_Trio doing a happy spin in the Prism Plains"* — giving immediate feedback on whether the blue wave marking stays geometrically stable.

How I Use AI Agents in My Daily Workflow

This is the part I'm most excited about. AI agents aren't just the product here — they're how I build the product.

1. Claude Code as a Development Partner

I use Claude Code (Anthropic's CLI coding agent) as a persistent development collaborator across the entire project. It's not a code-completion tool — it's an agent with context. I've built a structured planning framework called GSD that Claude Code understands natively. I issue commands like `/gsd-plan-phase` and it reads the project state, understands what milestone we're on, and drafts implementation plans against the formal requirements.

2. Agent Rules as Executable Infrastructure Knowledge

I wrote a `.agents/rules/hermes_infrastructure.md` file that Claude Code loads automatically when working with infrastructure code. This means I don't have to re-explain the dual-VM architecture, the systemd naming conventions (`hermes-gateway.service`, not `hermes.service`), or the HuggingFace cache symlink requirements every session. The agent just knows. This has eliminated entire categories of mistakes — no more accidental recursive syncs that blow up the VM, no more leaving the GPU running overnight.

3. Multi-Agent Development Sessions

When I'm working on complex features that span multiple domains, I spawn parallel Claude Code agents. For example, when building the review routing engine (Phase 5), one agent researched the Hermes cron documentation while another audited the Slack OAuth scope requirements. They worked concurrently, and I synthesized the results. This turns what would be a 4-hour research session into 20 minutes.

4. The Agents Build the Agents

The custom Hermes plugins (`monstoryx-visual`, `monstoryx-drive`) were co-developed with Claude Code. I described the tool signatures I needed ("a tool that generates images via Nano Banana API and routes skill docs based on HERMES_PROFILE"), and the agent scaffolded the plugin structure, registered the tools, handled the environment variable routing, and wrote the SKILL.md extraction hooks. I then reviewed, tested, and deployed.

5. Continuous Context via Memory

I use Honcho (an agent memory layer) so that Hermes profiles retain context across sessions. The Deriver agent extracts observations from every interaction, the Dialectic agent retrieves relevant memory when answering queries, and the Dreamer agent consolidates and improves memory quality during idle time. This means the Creative agent remembers which storyboard panel was approved last week without me re-prompting it.

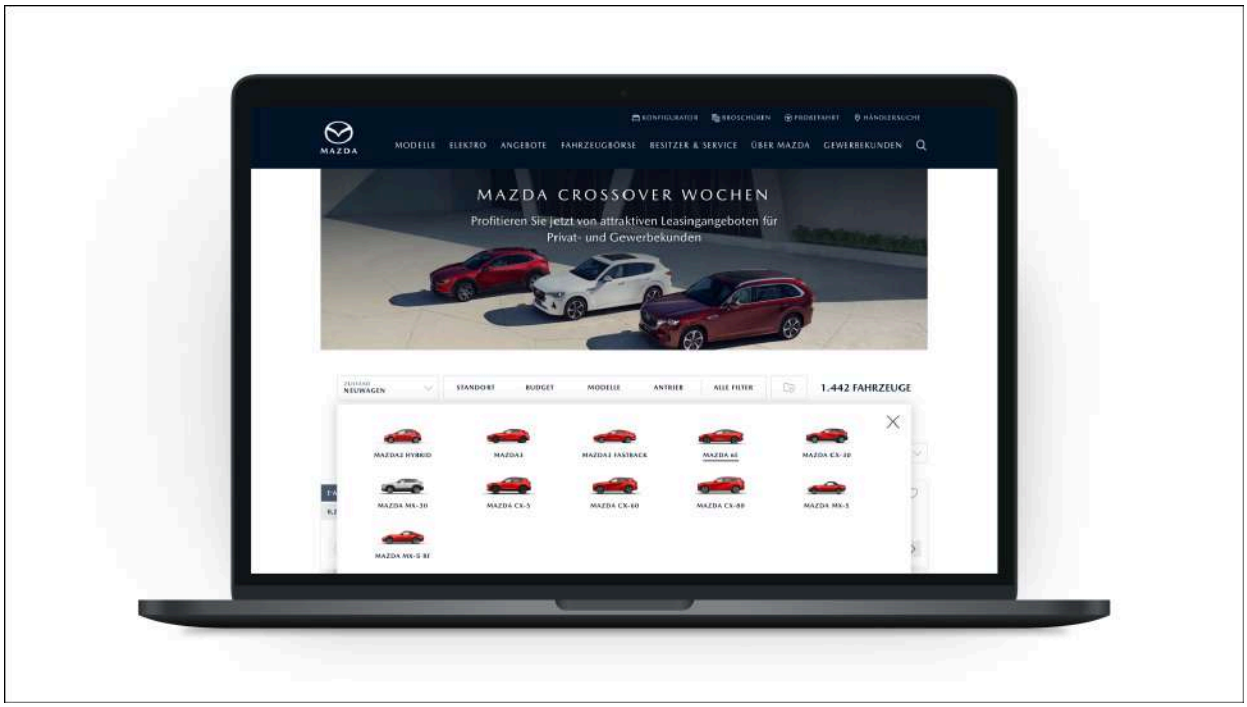
Results & What I Learned

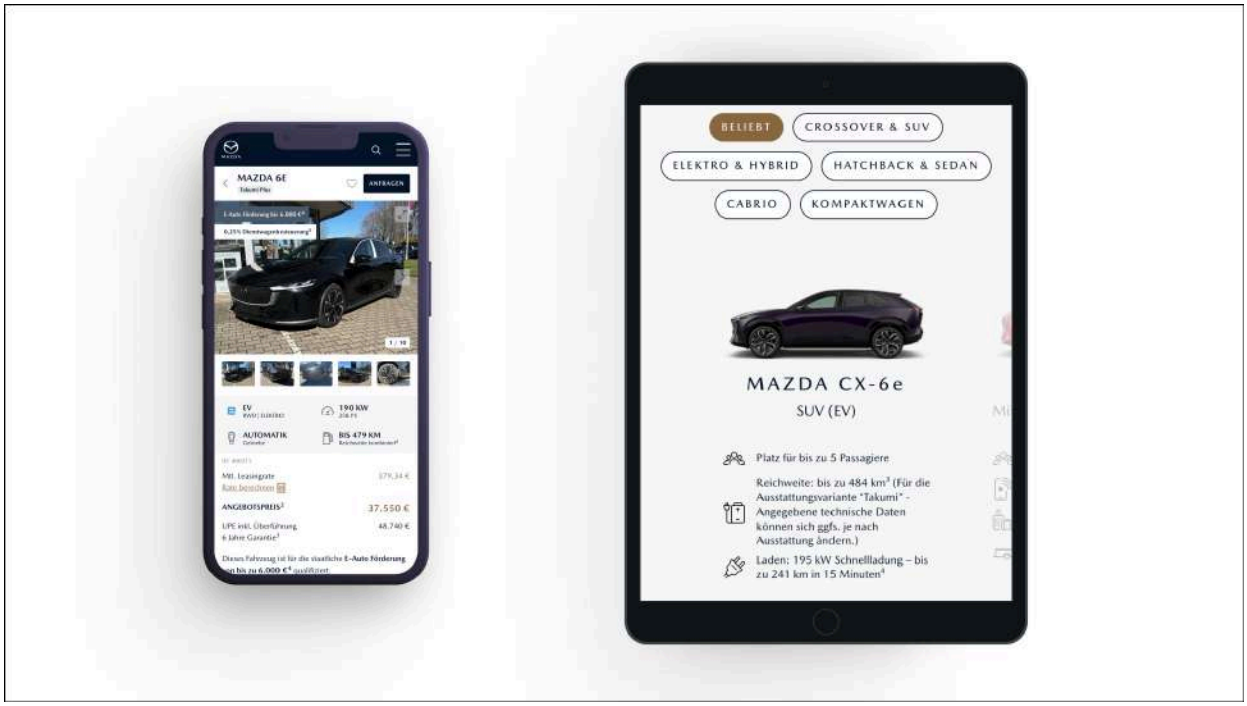
- **Shipped 4 milestones in ~4 weeks** (v1.0 through v1.3 partial), each containing multiple phases with formal UAT validation
- **GPU costs reduced by ~95%** through the serverless architecture pattern
- **Zero production incidents** on the multi-agent system thanks to the agent rules file preventing infrastructure mistakes
- **Key insight:** The bottleneck in agent systems isn't model quality — it's engineering discipline around state management, error recovery, and human-in-the-loop timeout handling (Slack's 3-second timeout is brutal if you don't `ack()` first)

The meta-lesson: **use agents to build agents**. Every time I learn something about the infrastructure, I encode it into a rule file so the development agent never makes that mistake again. The system gets smarter as I work on it.

SELECTED WORKS

My Mazda App






FINANZIERUNGSRECHNER
LEASING
VARIO OPTION
CLASSIC
Monatliche Rate
379,34 €
Nettodarlehensbetrag
36.772,00 €
Gesamtbetrag
15.300,96 €


MEHR ERFAHREN
MAZDA6e
AUF IHRE BEDÜRFNISSE AUSGERICHTET
JETZT WEITER KONFIGURIEREN

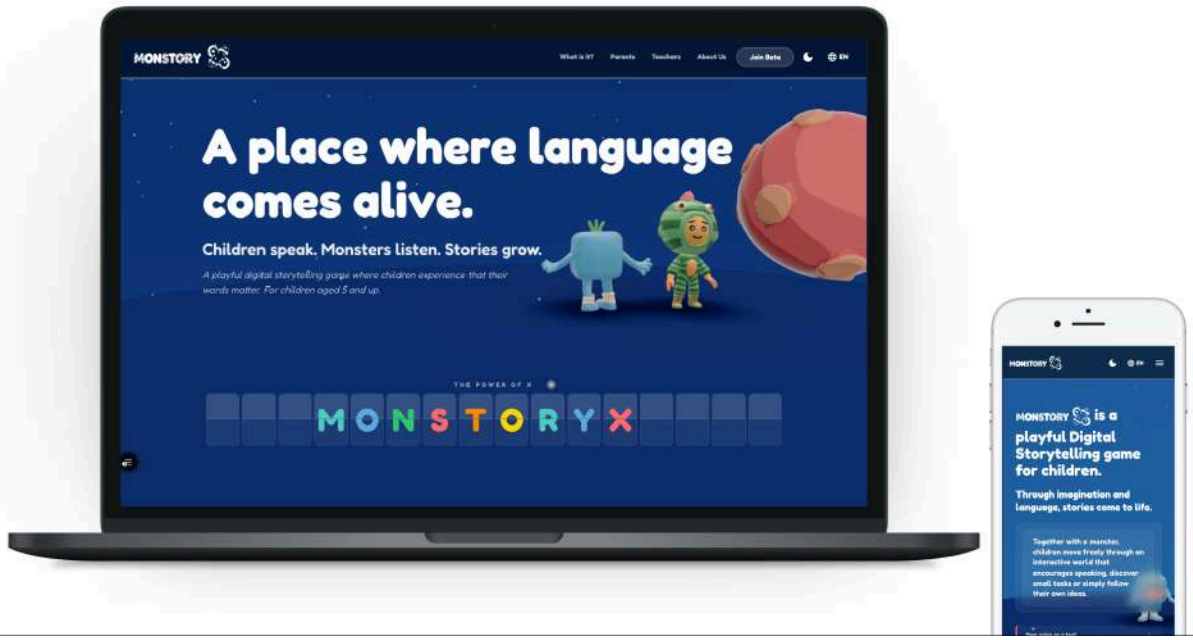

GLOBAL
INNOVATION
SUMMIT
EXHIBITION

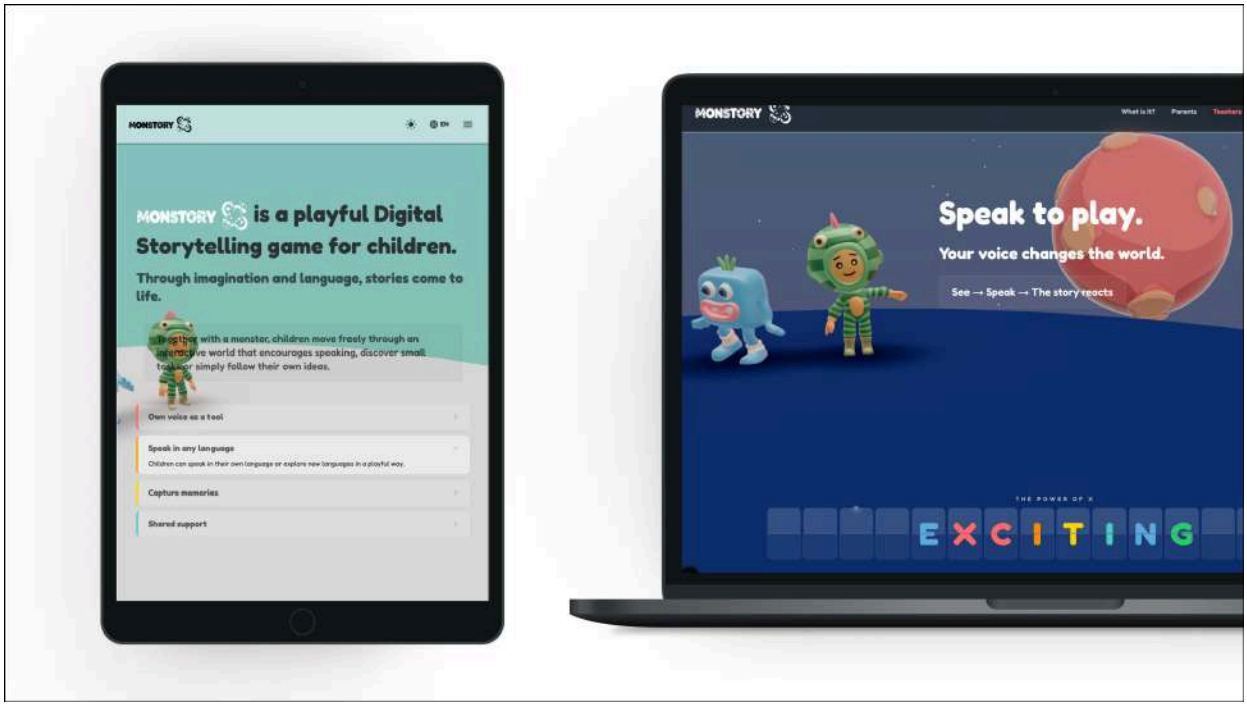

GLOBAL
INNOVATION
SUMMIT
EXHIBITION
Green Transition
Post COVID
Digital
Transformation
Austria
Austria
Show More


GLOBAL
INNOVATION
SUMMIT
EXHIBITION
About this Exhibition
Index
Guided Tour
Innovations
Green Transition
Post COVID
Digital
Transformation
Countries
Organisations
Imprint
Privacy Policy
Austria
Austria
About Us
Our Focus
More Information
Get in touch via #G2S2021 B2B meeting

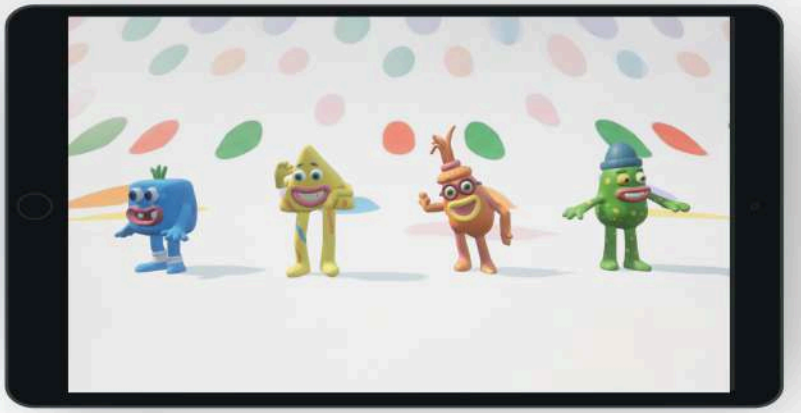

GLOBAL
INNOVATION
SUMMIT
EXHIBITION
About this Exhibition
Index
Guided Tour
Innovations
Green Transition
Post COVID
Digital
Transformation
Countries
Organisations
Imprint
Privacy Policy
"Humans spend more than 90% of their lives in building. Let's make sure they are safe."
COVID-19-Free Buildings
Show More


GLOBAL
INNOVATION
SUMMIT
EXHIBITION
About this Exhibition
Index
Guided Tour
Innovations
Green Transition
Post COVID
Digital
Transformation
Countries
Organisations
Imprint
Privacy Policy
Eureka Clusters
Show More
Virtual Exhibition | Organisations
Eureka Clusters
About Us
CELTIC-NEXT - EUROGIA - ITEA - PENTA-EURIPIDES³ - SMART
The Eureka Clusters are industry-driven R&D programmes with thematic communities of experts. They stimulate pan-European collaboration between large companies, SMEs, RTOs and universities.
Eureka Cluster projects are:
Industry-driven, supported by national funding bodies
Bottom-up
Market-oriented
Open to global cooperation
Cover the full value chain to create innovative solutions
share

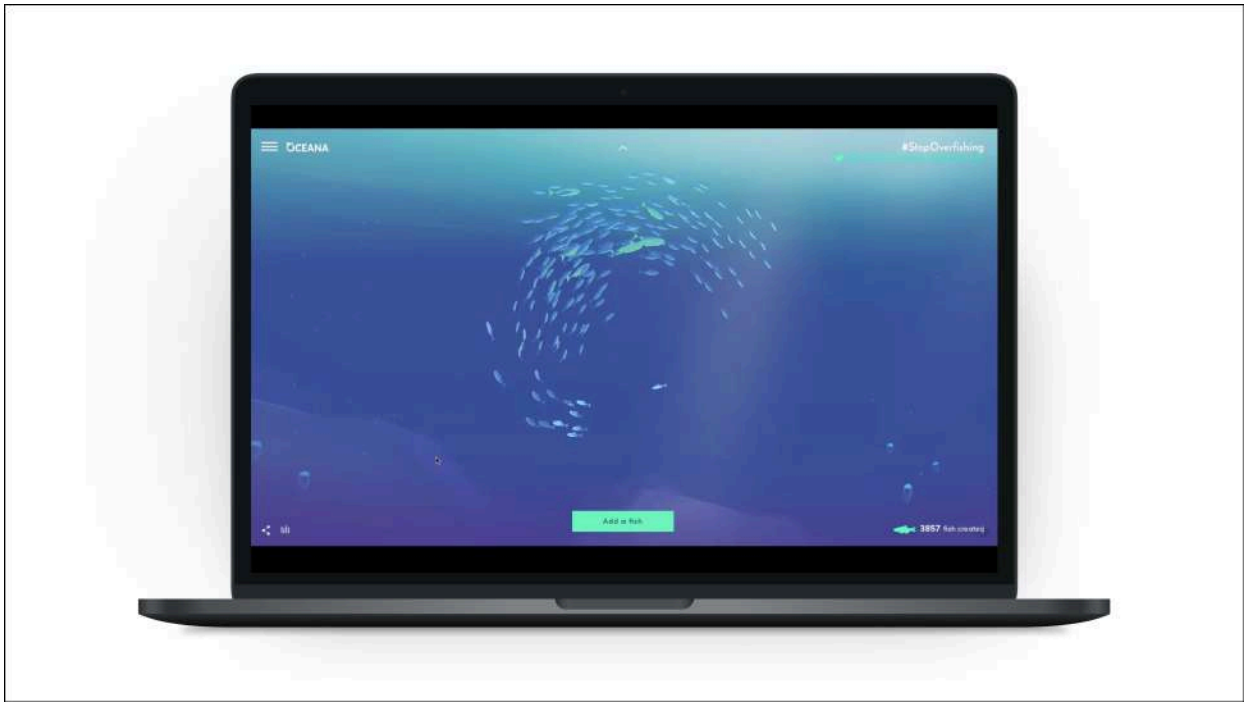

MONSTORY
What is it?
Parents
Teachers
About Us
Join Beta
EN
A place where language comes alive.
Children speak. Monsters listen. Stories grow.
A playful digital storytelling game where children experience that their words matter. For children aged 5 and up.
THE POWER OF X
MONSTORYX
MONSTORY is a playful Digital Storytelling game for children.
Through imagination and language, stories come to life.

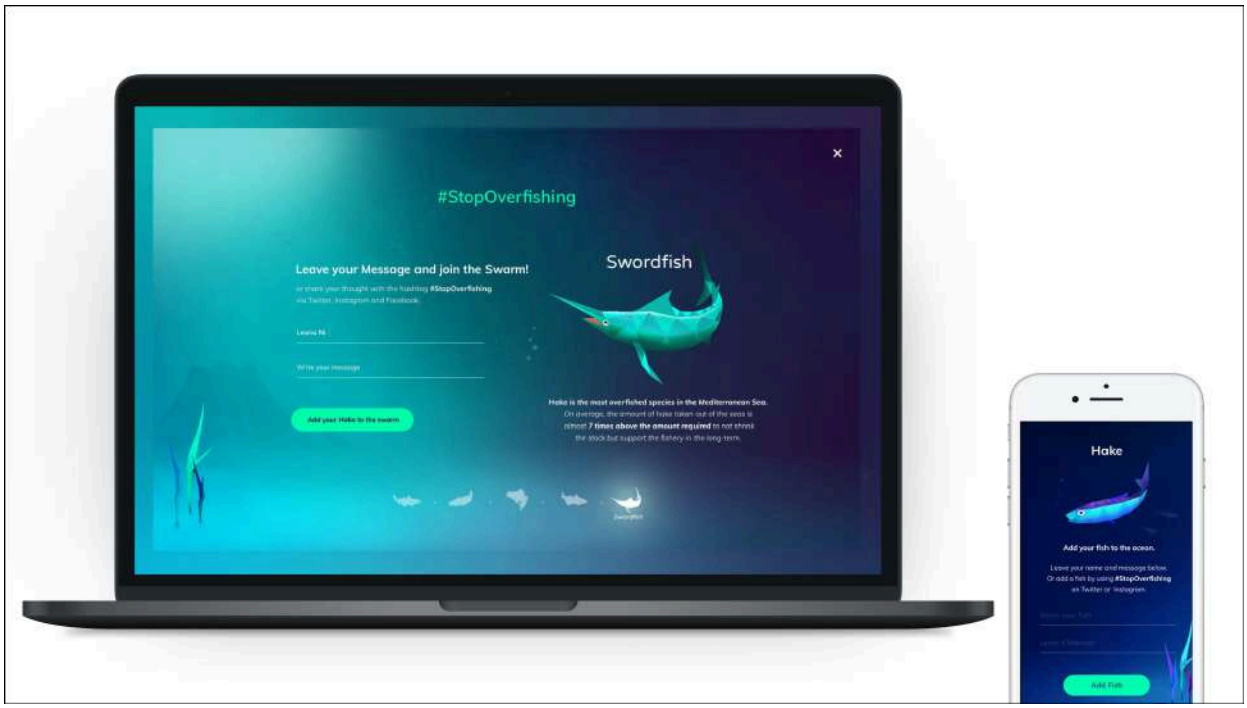

MONSTORY
MONSTORY is a playful Digital Storytelling game for children.
Through imagination and language, stories come to life.
Starting with a monster, children move freely through an interactive world that encourages speaking, discover small tasks or simply follow their own ideas.
Own voice as a tool
Speak in any language
Children can speak in their own language or explore new languages in a playful way.
Capture memories
Shared support
MONSTORY
What is it?
Parents
Speak to play.
Your voice changes the world.
See → Speak → The story reacts
THE POWER OF X
EXCITING

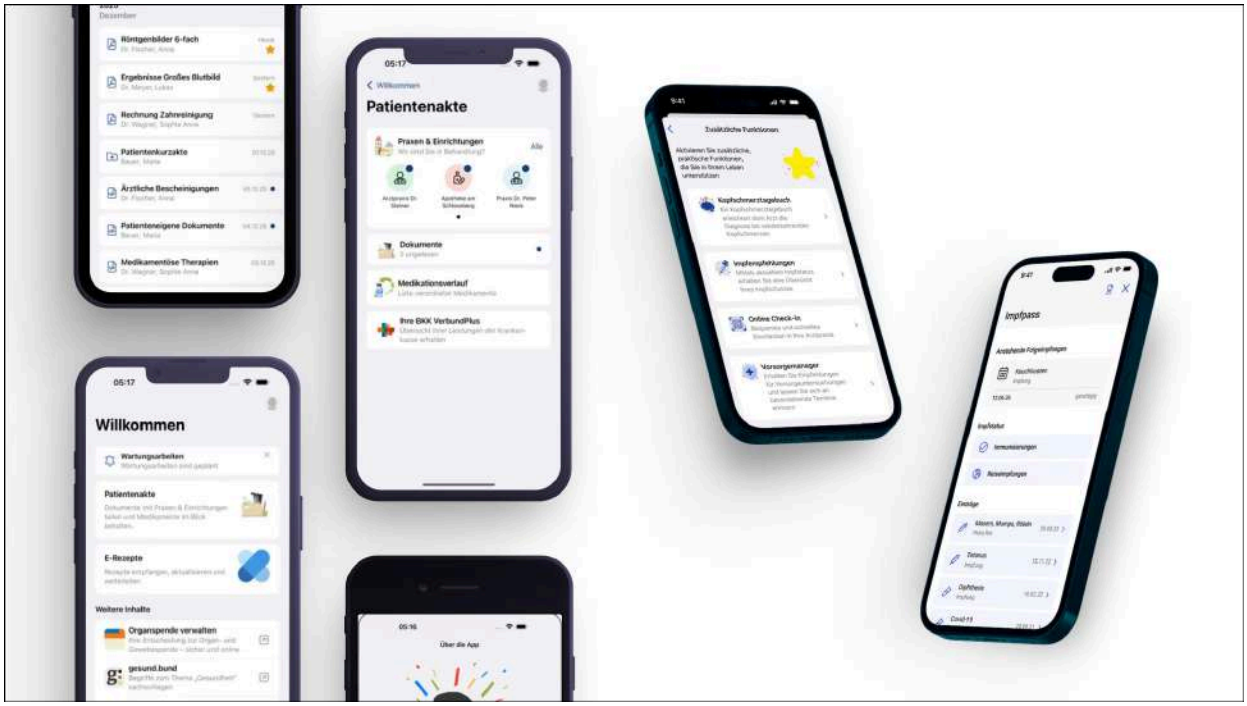

MONSTORY

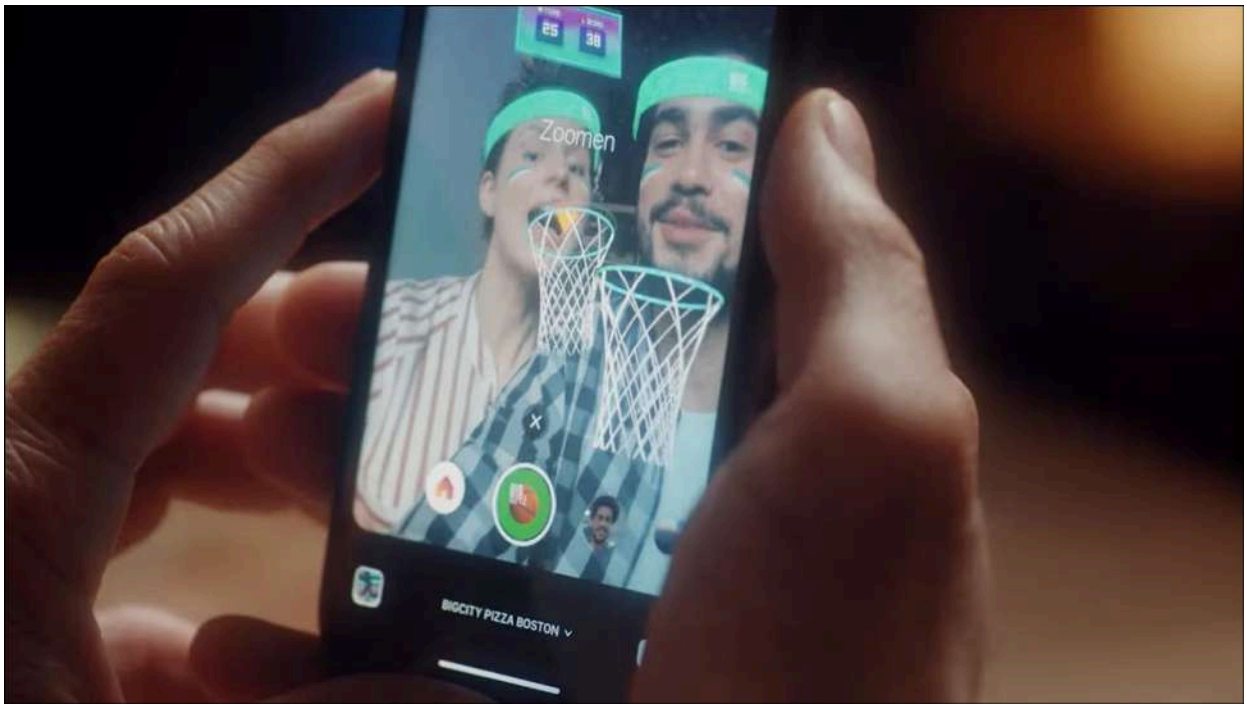


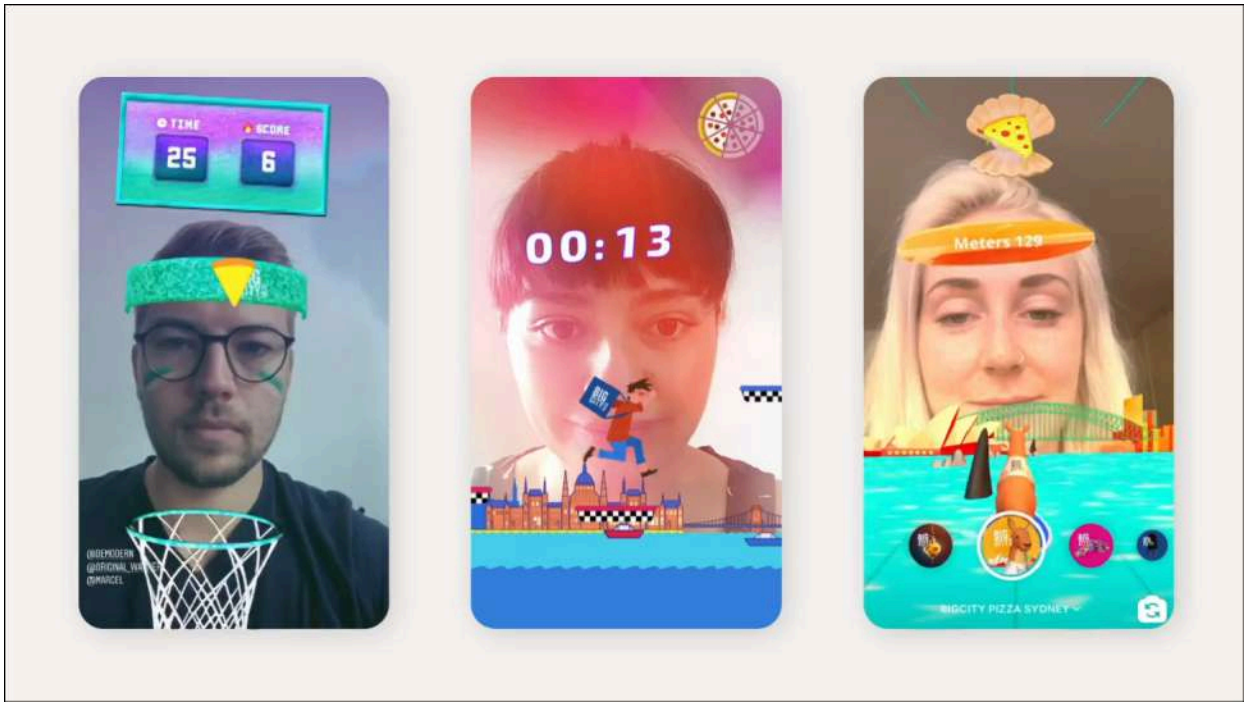

1630
0
REC


OCEANA
#StopOverfishing
Add a fish


#StopOverfishing
Leave your Message and join the Swarm!
Swordfish
Add your Hake to the swarm
Hake
Add your fish to the ocean.

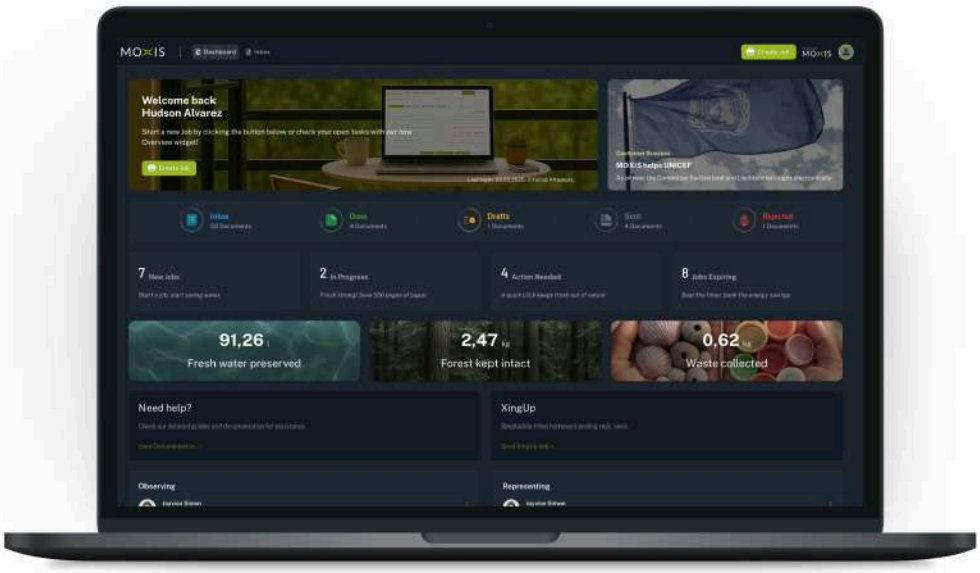

Röntgenbilder 6-fach
Ergebnisse Großes Blutbild
Rechnung Zahnreinigung
Patientenkurzakte
Ärztliche Bescheinigungen
Patienteneigene Dokumente
Medikamentöse Therapien
Patientenakte
Praxen & Einrichtungen
Dokumente
Medikationsverlauf
Ihre BKK VerbundPlus
Willkommen
Wartungsarbeiten
Patientenakte
E-Rezepte
Weitere Inhalte
Organspende verwalten
gesund.bund
Zusätzliche Funktionen
Kopfschmerztagebuch
Impfempfehlungen
Online Check-In
Impfpass

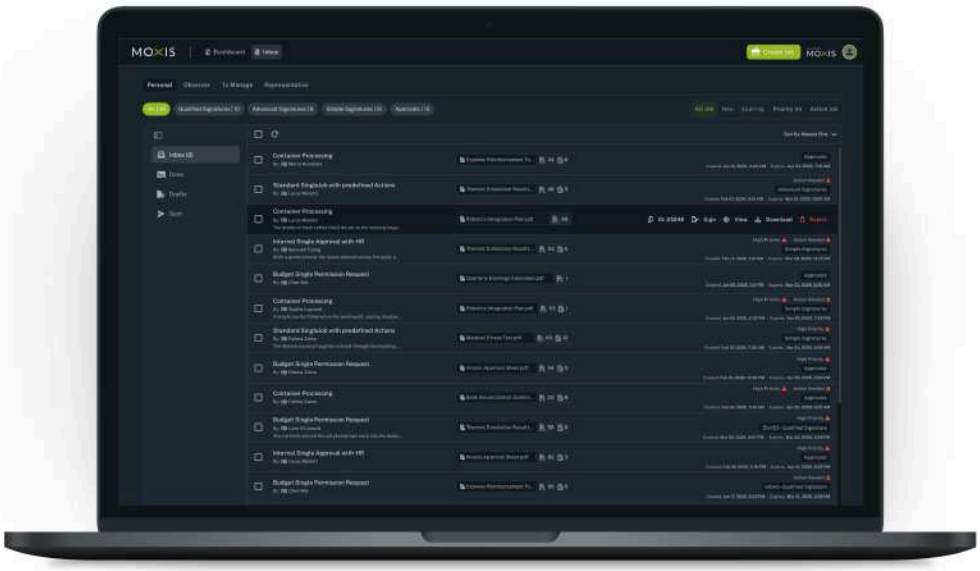

Zoomen
BIGCITY PIZZA BOSTON

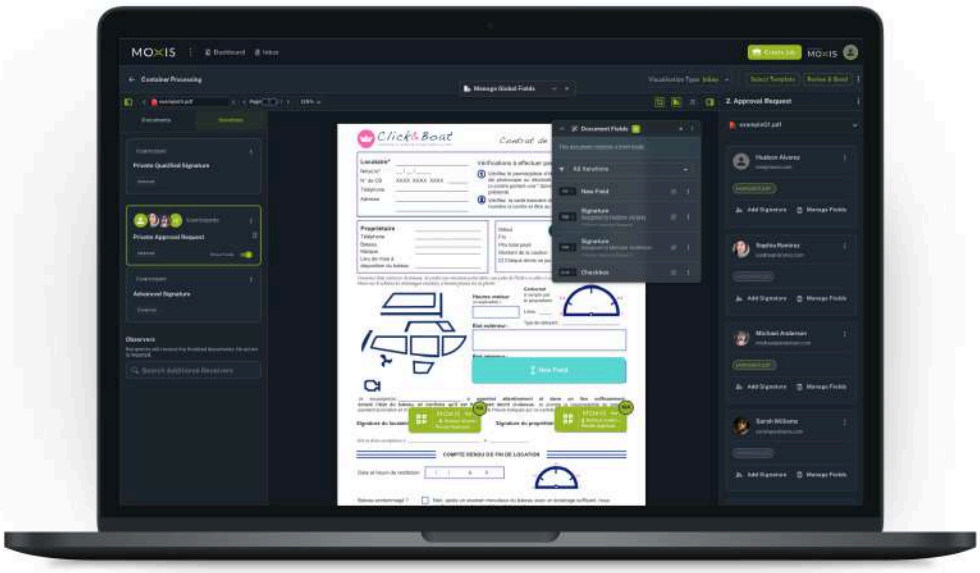

TIME
25
SCORE
6
00:13
Meters 129
BIGCITY PIZZA SYDNEY

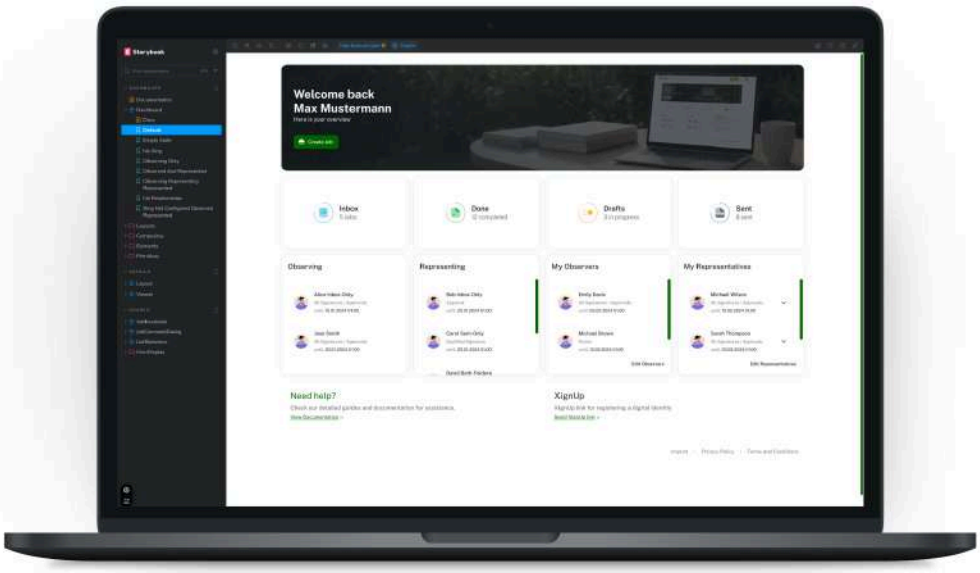

Traditionell trinken die Bären
welches Getränk?
Milch
Tee
Tomatensaft


WILLKOMMEN
NEUER
ERFINDER
SEBOOZLE
WEITER
Ich habe dir so viel zu erzählen. Womit wollen wir anfangen?
Ratespiel
Was ist dein Lieblingstier?
Australien ist berühmt für seine?
Kängurus
Erdbären
Hauskatzen
SPITZE, DU HAST DIR EINEN STICKER VERDIENT!
WOOZELIGER WITZBOLD


MOXIS
Welcome back Hudson Alvarez
MOXIS helps UNICEF
7 New Jobs
2 In Progress
4 Action Needed
8 Jobs Expiring
91,26 l
Fresh water preserved
2,47 kg
Forest kept intact
0,62 kg
Waste collected
Need help?
XingUp
Observing
Representing


MOXIS
Dashboard
Inbox
Create new
MOXIS
Personal
Observer
To Manage
Representative
All (24)
Qualified Signatures (4)
Advanced Signatures (8)
Simple Signatures (5)
Approvals (7)
All (24)
New
Expiring
Priority (4)
Action (6)
Sort By: Newest First
Inbox (8)
Done
Drafts
Sent
Container Processing
Standard Singlestep with predefined Actions
Container Processing
Internal Single Approval with HR
Budget Single Permission Request
Container Processing
Standard Singlestep with predefined Actions
Budget Single Permission Request
Container Processing
Budget Single Permission Request
Internal Single Approval with HR
Budget Single Permission Request
Sign
View
Download
Reject


MOXIS
Dashboard
Inbox
Container Processing
Private Qualified Signature
Private Approval Request
Advanced Signature
Observers
Search Additional Receivers
Click&Boat
Contrat de
Document Fields
All Sections
New Field
Signature
Checkbox
Approval Request
Add Signature
Manage Fields


Storybook
Welcome back
Max Mustermann
Here is your overview
Inbox
Done
Drafts
Sent
Observing
Representing
My Observers
My Representatives
Need help?
XignUp